

Linux Plumbers Conference

Dublin, Ireland September 12-14, 2022



LR/SC Architecture Forward Progress Guarantee

Guo Ren <guoren@kernel.org>

Linux Plumbers Conference | Dublin, Ireland Sept. 12-14, 2022

Background

Queued Spinlock is a standard implementation recommended by Linux, which is competent in an embedded SMP or a cloud server NUMA system. It solves the Fairness and Cache-line bouncing problems and ensures performance and size. RISC-V Linux has been trying to implement qspinlock since 2019, but it still hasn't been merged due to the Atomic (LR/SC) Forward Guarantee issue.

v1: Using cmpxchg implement xchg16 borrowed from MIPS (Michael Clark)
v2: Use LR/SC directly, and abandon cmpxchg in xchg16 (Guo Ren)
v3: Small fixup and coding convention
v4: Change to modify qspinlock xchg_tail without xchg16 (Suggested by Arnd, Rejected by Peterz)
Move to ticket-lock and the generic version wins. (riscv, openrisc, loongarch are using it)
v5: Add qspinlock with config switch of riscv/Kconfig
v6: Small fixups and coding convention
v7: Add combo-lock, one Image for both ticket-lock & qspinlock
v8: Add csky support ticket-lock & qspinlock
v9: Move csky & openrisc to support one qspinlock, and combo-lock for riscv

Agenda

- Atomic Instructions Category
- **Forward Progress Guarantee** concepts
- Axiom memory model
- Micro-arch cases
- ticket-lock & queued-spinlock
- Architectures summary



Atomic Instructions Category

AMO Instructions:

- atomic_op
- atomic_fetch_op
- atomic_op_return
- atomic_xchg
- atomic_cmpxchg (CAS)
- atomic_cmpxchg_dual (DCAS)

(op: and, sub, and, or, xor …)

Load-Reserved and Store-Conditional:

```
atomic_op:
        1:    LR        %tmp, (%addr)
              " #op "     %tmp, %val
              SC        %flag, %tmp, (%addr)
              bnez       %flag, 1b

atomic_fetch_op:
        1:    LR          %ret, (%addr)
              move         %tmp, %ret
              " #op "      %tmp, %val
              SC          %flag, %tmp (%addr)
              bnez         %flag, 1b
atomic_op_return …
xhg …
cmpxchg …
cmpxchg_dual … (need multiple src & dst regs)
```

Atomic Instructions Category

Pure LR/SC	arm32, csky, openrisc
Pure AMO	x86
Pure LR/SC or Pure AMO	arm64
Mixed AMO + LR/SC	riscv, loongarch



Why LR/SC?

4 advantages:

- CAS has ABA problem
- CAS is expensive in hw implementation
- DCAS is more expensive
- LR/SC semantic is more efficient than CAS

2 disadvantages:

- LR/SC causes livelock more than CAS
- DCAS is basic primitive in software ecosystem

Intel TSX is to solve multi-word atomic issues, but *TSX's fwd guarantee is weak and needs an alternative path for retry.*

RISC-V unprivileged ISA spec 8.2

Both compare-and-swap (CAS) and LR/SC can be used to build lock-free data structures. After extensive discussion, we opted for LR/SC for several reasons: 1) CAS suffers from the ABA problem, which LR/SC avoids because it monitors all accesses to the address rather than only checking for changes in the data value; 2) CAS would also require a new integer instruction format to support three source operands (address, compare value, swap value) as well as a different memory system message format, which would complicate microarchitectures; 3) Furthermore, to avoid the ABA problem, other systems provide a double-wide CAS (DW-CAS) to allow a counter to be tested and incremented along with a data word. This requires reading five registers and writing two in one instruction, and also a new larger memory system message type, further complicating implementations; 4) LR/SC provides a more efficient implementation of many primitives as it only requires one load as opposed to two with CAS (one load before the CAS instruction to obtain a value for speculative computation, then a second load as part of the CAS instruction to check if value is unchanged before updating).

The main disadvantage of LR/SC over CAS is livelock, which we avoid, under certain circumstances, with an architected guarantee of eventual forward progress as described below. Another concern is whether the influence of the current x86 architecture, with its DW-CAS, will complicate porting of synchronization libraries and other software that assumes DW-CAS is the basic machine primitive. A possible mitigating factor is the recent addition of transactional memory instructions to x86, which might cause a move away from DW-CAS.

More generally, a multi-word atomic primitive is desirable, but there is still considerable debate about what form this should take, and guaranteeing forward progress adds complexity to a system. Our current thoughts are to include a small limited-capacity transactional memory buffer along the lines of the original transactional memory proposals as an optional standard extension "T".

Forward Progress Guarantee

LR/SC forward progress guarantee: (atomic_op, cmpxchg)

```
1:   LR      %ret, (%addr)
     " #op " %ret, %val
     SC      %flag, %ret, (%addr)
     bnez    %flag, 1b
```

Only local events could fail conditional store

CAS loop forward progress guarantee:

```
1:   cas     %ret, %old, %new, (%addr)
     move    %tmp, %old
     move    %old, %ret
     bneq    %ret, %tmp, 1b
```

The second loop always breaks without local events

```
static __always_inline u32 xchg_tail(struct qspinlock *lock, u32 tail)
{
        u32 old, new, val = atomic_read(&lock->val);

        for (;;) {
                new = (val & _Q_LOCKED_PENDING_MASK) | tail;
                /*
                 * We can use relaxed semantics since the caller ensures that
                 * the MCS node is properly initialized before updating the
                 * tail.
                 */
                old = atomic_cmpxchg_relaxed(&lock->val, val, new);
                if (old == val)
                        break;

                val = old;
        }
        return old;
}
```

OS Axiom Memory Model (LKMM)

herd7 .cat:
https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/tools/memory-model
litmus-cases:
https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/tools/memory-model/litmus-tests

No LR/SC primitives were found, all are high-level abstractions:

- READ_ONCE(*x), WRITE_ONCE(*x, 1)
- smp_mb(); smp_rmb(); smp_wmb();
- smp_load_acquire(y); smp_store_release(y, 1);
- spin_is_locked(lo); spin_is_unlocked(lo);
- rcu_read_lock(); rcu_dereference(*p); rcu_read_unlock();

ISA Axiom Memory Model (riscv)

herd7 .cat:
https://github.com/riscv/riscv-isa-manual/blob/master/src/memory-model-herd.tex
litmus-cases:
https://github.com/litmus-tests/litmus-tests-riscv

```
RISCV SWAP-LR-SC+FULL
{
0:x5=x;
1:x5=x;
}
 P0                  | P1                  ;
 ori x6,x0,1         | ori x6,x0,2         ;
 lr.w x7,0(x5)       | lr.w x7,0(x5)       ;
 sc.w x8,x6,0(x5)    | sc.w x8,x6,0(x5)    ;

forall
(x=1 /\ 1:x7=0 /\ 0:x7=2) \/
(0:x7=0 /\ x=2 /\ 1:x7=1) \/
(not (0:x8 = 0) \/ not (1:x8 = 0))
```

SWAP-LR-SC on U540

https://raw.githubusercontent.com/litmus-tests/litmus-tests-riscv/master/hw-results/SiFive-Freedom-U540.log

```
{
0:x5=x;
1:x5=x;
}
 P0                 | P1                 ;
 ori x6,x0,1        | ori x6,x0,2        ;
 lr.w x7,0(x5)      | lr.w x7,0(x5)      ;
 sc.w x8,x6,0(x5)   | sc.w x8,x6,0(x5)   ;
```

```
Test SWAP-LR-SC+FULL Require
Histogram (6 states)
663      :> 0:x7=0; 0:x8=0; 1:x7=0; 1:x8=1; x=1;
600165150:> 0:x7=0; 0:x8=0; 1:x7=1; 1:x8=0; x=2;
144      :> 0:x7=0; 0:x8=0; 1:x7=1; 1:x8=1; x=1;
318      :> 0:x7=0; 0:x8=1; 1:x7=0; 1:x8=0; x=2;
600033478:> 0:x7=2; 0:x8=0; 1:x7=0; 1:x8=0; x=1;
247      :> 0:x7=2; 0:x8=1; 1:x7=0; 1:x8=0; x=2;
Ok
Witnesses
Positive: 1200200000 Negative: 0
Condition forall (x=1 /\ 1:x7=0 /\ 0:x7=2 \/ 0:x7=0 /\ x=2 /\ 1:x7=1 \/ not (0:x8=0) \/ not (1:x8=0)) is validated
Hash=a8cf3f2bc46a0f7ca32246ac29f09572
Time SWAP-LR-SC+FULL 396.02
```

0.00000114314 (millionth failed)
strong forward progress guarantee

SWAP-LR-SC on QEMU

```
Test SWAP-LR-SC+FULL Required
Histogram (4 states)
9997839:>0:x7=2; 0:x8=0; 1:x7=0; 1:x8=0; x=1;
1427  :>0:x7=0; 0:x8=0; 1:x7=0; 1:x8=1; x=1;
1768  :>0:x7=0; 0:x8=1; 1:x7=0; 1:x8=0; x=2;
9998966:>0:x7=0; 0:x8=0; 1:x7=1; 1:x8=0; x=2;
Ok

Witnesses
Positive: 20000000, Negative: 0
Condition forall (x=1 /\ 1:x7=0 /\ 0:x7=2 \/ 0:x7=0 /\ x=2 /\ 1:x7=1 \/ not (0:x8=0) \/ not (1:x8=0)) is validated
Hash=a8cf3f2bc46a0f7ca32246ac29f09572
Observation SWAP-LR-SC+FULL Always 20000000 0
Time SWAP-LR-SC+FULL 209.63
```

```
{
0:x5=x;
1:x5=x;
}
 P0                 | P1                 ;
 ori x6,x0,1        | ori x6,x0,2        ;
 lr.w x7,0(x5)      | lr.w x7,0(x5)      ;
 sc.w x8,x6,0(x5)   | sc.w x8,x6,0(x5)   ;
```

0.000159749994 (one ten thousandth failed)
100 times weaker

SWAP-LR-SC (2) on QEMU

```
RISCV SWAP-LR-SC+FULL
{
0:x5=x;
1:x5=x;
}
 P0                 | P1                 ;
 ori x6,x0,1        | ori x6,x0,2        ;
 ori x6,x0,1        | sw x6, 0(x5)       ;
 ori x6,x0,1        | sw x6, 0(x5)       ;
... (rept 100)
 lr.w x7,0(x5)      | sw x6, 0(x5)       ;
 ori x6,x0,1        | sw x6, 0(x5)       ;
 ori x6,x0,1        | sw x6, 0(x5)       ;
 ori x6,x0,1        | sw x6, 0(x5)       ;
 sc.w x8,x6,0(x5)   | sw x6, 0(x5)       ;
 ori x6,x0,1        | sw x6, 0(x5)       ;
 ... (rept 100)

forall
((0:x8 = 0))
```

```
Test SWAP-LR-SC+FULL Required
Histogram (2 states)
197719:>0:x8=0;
2281  *>0:x8=1;
No

Witnesses
Positive: 197719, Negative: 2281
Condition forall (0:x8=0) is NOT validated
Hash=f5d750cc0f60c208b752b565711cf7d1
Observation SWAP-LR-SC+FULL Sometimes 197719 2281
Time SWAP-LR-SC+FULL 2.10
Revision exported, version 7.56.2
```

0.0115365749 (one percent)
another 100 times weaker



Linux Plumbers Conference | Dublin, Ireland Sept. 12-14, 2022

SC result:
0 -> success
1 -> fail by local event
2 -> fail by remote event (Never)

Micro-arch case: Exclusive Monitors

Armv6: An exclusive monitor is a simple state machine, with the possible states open and exclusive.A Load-Exclusive operation updates the monitors to exclusive state. A Store-Exclusive operation accesses the monitor(s) to determine whether it can complete successfully. A Store-Exclusive can succeed only if all accessed exclusive monitors are in the exclusive state.
https://developer.arm.com/documentation/dht0008/a/arm-synchronization-primitives/exclusive-accesses/exclusive-monitors

```
#define ATOMIC_OP(op, c_op, asm_op)
static inline void atomic_##op(int i, atomic_t *v)
{
        unsigned long tmp;
        int result;

        prefetchw(&v->counter);
        __asm__ __volatile__("@ atomic_" #op "\n"
"1:     ldrex   %0, [%3]\n"
"       " #asm_op "     %0, %0, %4\n"
"       strex   %1, %0, [%3]\n"
"       teq     %1, #0\n"
"       bne     1b"
        : "=&r" (result), "=&r" (tmp), "+Qo" (v->counter)
        : "r" (&v->counter), "Ir" (i)
        : "cc");
}
```

```
ARM: atomics: prefetch the destination word for write prior to strex

The cost of changing a cacheline from shared to exclusive state can be
significant, especially when this is triggered by an exclusive store,
since it may result in having to retry the transaction.

This patch prefixes our atomic access implementations with pldw
instructions (on CPUs which support them) to try and grab the line in
exclusive state from the start. Only the barrier-less functions are
updated, since memory barriers can limit the usefulness of prefetching
data.

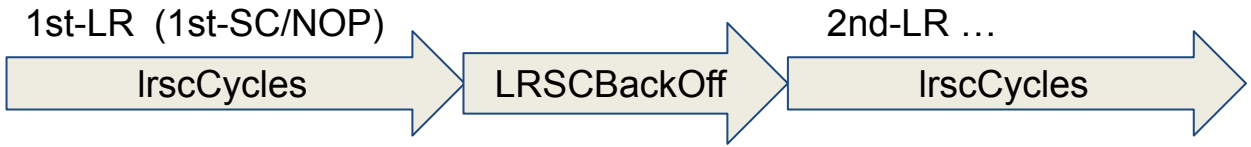
Acked-by: Nicolas Pitre <nico@linaro.org>
Signed-off-by: Will Deacon <will.deacon@arm.com>
```

Micro-arch case: lock + backoff (LR/SC Fwd guarantee)

After the LR is executed, the processor will enter the following states in sequence:

1. Block all probe requests to the current core's cache line and block the execution of subsequent LR for tens of cycles. (lrscCycles - LRSCBackOff cycles) unless meet SC.
2. Continue to block the execution of subsequent LR, but allow probe requests to this cache line from other CPUs for LRSCBackOff cycles. (LR BackOff)
3. Return to normal

ref: https://xiangshan-doc.readthedocs.io/zh_CN/latest/memory/fu/atom/

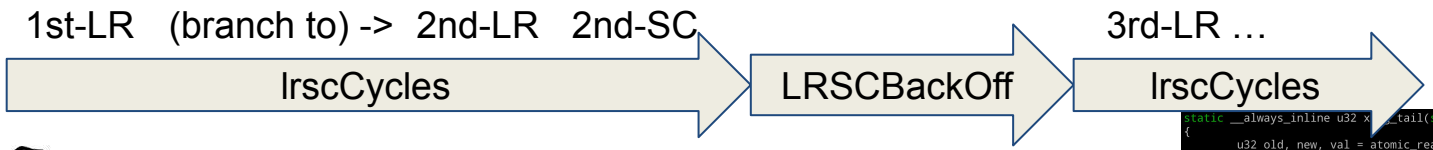


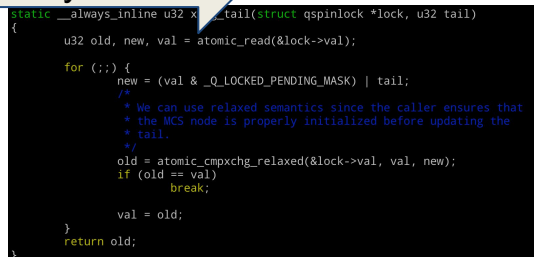
It only solve LR/SC Fwd guarantee, how about CAS Fwd guarantee?

Micro-arch case: lock + backoff (cmpxchg loop Fwd guarantee)

After the LR is executed, the processor will enter the following states in sequence:

1. Block all probe requests to the current core's cache line and block the execution of subsequent LR for lrscCycles - LRSCBackOff cycles (tens of cycles)
2. Second LR would reset the lrscCycles counter, then it would continue to block all remote probe requests and block the execution of subsequent LR for lrscCycles - LRSCBackOff cycles. The second SC can't break it again.
3. Continue to block the execution of subsequent LR, but allow probe requests to this cache line from other CPUs for LRSCBackOff cycles.
4. After LRSCBackOff cycles pass, it returns to normal.





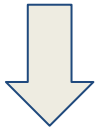
```
static __always_inline u32 x
_tail(struct qspinlock *lock, u32 tail)
{
        u32 old, new, val = atomic_read(&lock->val);

        for (;;) {
                new = (val & _Q_LOCKED_PENDING_MASK) | tail;
                /*
                 * We can use relaxed semantics since the caller ensures that
                 * the MCS node is properly initialized before updating the
                 * tail.
                 */
                old = atomic_cmpxchg_relaxed(&lock->val, val, new);
                if (old == val)
                        break;

                val = old;
        }
        return old;
}
```

Micro-arch case: Atomic map

```
Load-Reserved and Store-Conditional:
atomic_op:
        1:    LR        %tmp, (%addr)
              " #op "     %tmp, %val
              SC        %flag, %tmp, (%addr)
              bnez       %flag, 1b
```

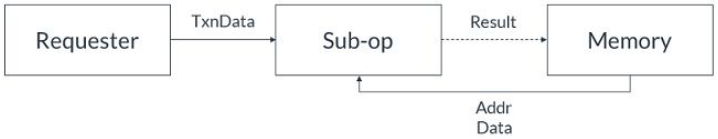


Macro-op Fusion

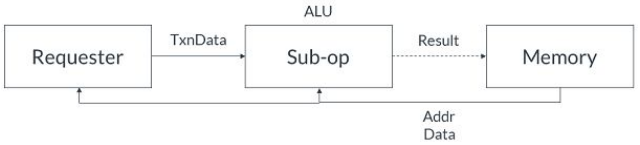
Micro-arch AMO Operations

Micro-arch case: Atomic map (AMBA CHI Atomic Transaction)

AtomicStore:



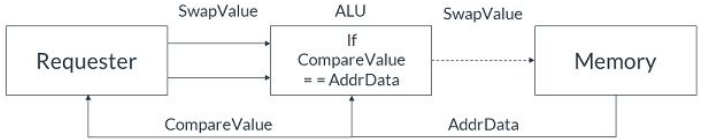
AtomicLoad:



AtomicSwap:



AtomicCompare:



https://developer.arm.com/documentation/102714/0100/Transaction-types

LR/SC depends on micro-arch fwd guarantee

```
diff --git a/include/asm-generic/qspinlock.h b/include/asm-generic/qspinlock.h
index 995513fa2690..0000ea26e328 100644
--- a/include/asm-generic/qspinlock.h
+++ b/include/asm-generic/qspinlock.h
@@ -15,8 +15,8 @@
  * qspinlock relies on a far greater (compared to asm-generic/spinlock.h) set
  * of atomic operations to behave well together, please audit them carefully to
  * ensure they all have forward progress. Many atomic operations may default to
- * cmpxchg() loops which will not have good forward progress properties on
- * LL/SC architectures.
+ * cmpxchg() loops which needs LL/SC architectures have a good forward progress
+ * guarantee.
  *
  * One notable example is atomic_fetch_or_acquire(), which x86 cannot (cheaply)
  * do. Carefully read the patches that introduced
~
```

ticket-lock & queued-spinlock

'Generic' ticket-lock implementation (comments in include/asm-generic/spinlock.h)

- It relies on **atomic_fetch_add()** having well defined forward progress guarantees under contention. If your architecture cannot provide this, stick to a test-and-set lock.

Queued spinlock (comments in include/asm-generic/qspinlock.h)

- qspinlock relies on a far greater (compared to asm-generic/spinlock.h) set of atomic operations to behave well together, please audit them carefully to ensure they all have forward progress. Many atomic operations may default to cmpxchg() loops which needs LL/SC architectures have a good forward progress guarantee.
- qspinlock also heavily relies on mixed size atomic operations, in specific it requires architectures to have **xchg16**; something which many LL/SC architectures need to implement as a 32bit and+or in order to satisfy the forward progress guarantees mentioned above.

Architectures Summary (arm32, csky, openrisc)

They are pure LR-SC architectures, that means all atomic operations are composed by LR-SC instructions paris. If LR-SC fwd is not guaranteed, it would affect the whole OS atomic primitives.

arm32	ticket-lock
csky	qspinlock
openrisc	ticket-lock



Architectures Summary (loongarch)

```
LoongArch: Add subword xchg/cmpxchg emulation

LoongArch only support 32-bit/64-bit xchg/cmpxchg in native. But percpu
operation, qspinlock and some drivers need 8-bit/16-bit xchg/cmpxchg. We
add subword xchg/cmpxchg emulation in this patch because the emulation
has better performance than the generic implementation (on NUMA system),
and it can fix some build errors meanwhile [1].

LoongArch's guarantee for forward progress (avoid many ll/sc happening
at the same time and no one succeeds):

We have the "exclusive access (with timeout) of ll" feature to avoid
simultaneous ll (which also blocks other memory load/store on the same
address), and the "random delay of sc" feature to avoid simultaneous
sc. It is a mandatory requirement for multi-core LoongArch processors
to implement such features, only except those single-core and dual-core
processors (they also don't support multi-chip interconnection).

Feature bits are introduced in CPUCFG3, bit 3 and bit 4 [2].

[1] https://lore.kernel.org/loongarch/CAAhV-H6vvkuOzy8OemWdYK3taj5Jn3bFXOZTwE=twM8ywpBUYA@mail.gmail.com/T/#t
[2] https://loongson.github.io/LoongArch-Documentation/LoongArch-Vol1-EN.html#_cpucfg

Reported-by: Sudip Mukherjee (Codethink) <sudipm.mukherjee@gmail.com>
Suggested-by: Linus Torvalds <torvalds@linux-foundation.org>
Signed-off-by: Rui Wang <wangrui@loongson.cn>
Signed-off-by: Huacai Chen <chenhuacai@loongson.cn>
```

dual-core: ticket-lock
others: qspinlock

Architectures Summary (riscv)

8.3 Eventual Success of Store-Conditional Instructions

As a consequence of the eventuality guarantee, if some harts in an execution environment are executing constrained LR/SC loops, and no other harts or devices in the execution environment execute an unconditional store or AMO to that reservation set, then at least one hart will eventually exit its constrained LR/SC loop. By contrast, if other harts or devices continue to write to that reservation set, it is not guaranteed that any hart will exit its LR/SC loop.

```
On Mon, Mar 29, 2021 at 08:01:41PM +0800, Guo Ren wrote:
> u32 a = 0x55aa66bb;
> u16 *ptr = &a;
>
> CPU0                        CPU1
> =========                =========
> xchg16(ptr, new)      while(1)
>                                     WRITE_ONCE(*(ptr + 1), x);
>
> When we use lr.w/sc.w implement xchg16, it'll cause CPU0 deadlock.

Then I think your LL/SC is broken.

That also means you really don't want to build super complex locking
primitives on top, because that live-lock will percolate through.

Step 1 would be to get your architecute fixed such that it can provide
fwd progress guarantees for LL/SC. Otherwise there's absolutely no point
in building complex systems with it.
```



ticket-lock & qspinlock for riscv

```
TYPE                    : COMBO | TICKET | QUEUED
arch_spin_lock          : 106   | 60     | 50
arch_spin_unlock        : 54    | 36     | 26
arch_spin_trylock       : 110   | 72     | 54
arch_spin_is_locked     : 48    | 34     | 20
arch_spin_is_contended  : 56    | 40     | 24
rch_spin_value_unlocked : 48    | 34     | 24

One example of disassemble combo arch_spin_unlock:
   0xffffffff8000409c <+14>:    nop # jump label slot
   0xffffffff800040a0 <+18>:    fence   rw,w # queued spinlock start
   0xffffffff800040a4 <+22>:    sb      zero,0(a4) # queued spinlock end
   0xffffffff800040a8 <+26>:    ld      s0,8(sp)
   0xffffffff800040aa <+28>:    addi    sp,sp,16
   0xffffffff800040ac <+30>:    ret
   0xffffffff800040ae <+32>:    lw      a5,0(a4) # ticket spinlock start
   0xffffffff800040b0 <+34>:    sext.w  a5,a5
   0xffffffff800040b2 <+36>:    fence   rw,w
   0xffffffff800040b6 <+40>:    addiw   a5,a5,1
   0xffffffff800040b8 <+42>:    slli    a5,a5,0x30
   0xffffffff800040ba <+44>:    srli    a5,a5,0x30
   0xffffffff800040bc <+46>:    sh      a5,0(a4) # ticket spinlock end
   0xffffffff800040c0 <+50>:    ld      s0,8(sp)
   0xffffffff800040c2 <+52>:    addi    sp,sp,16
   0xffffffff800040c4 <+54>:    ret
```

Architectures Summary (loongarch, riscv)

Ticket-lock (supported by ISA):

- They all have 32-bit AMO ADD instructions, so they satisfy the **ticket-lock**'s atomic_fetch_add() requirement.

qspinlock (supported by micro-arch):

- Both of them need micro-arch to support cmpxchg loop forward progress guarantee.
- Both of them need 32bit LR/SC instructions to implement xchg16.

Appendix
(multi-word atomic primitive)

- Support cmpxchg_dual
- Support atomic64_op for 32BIT

Dual elements with strong fwd guaranteeing is enough. If more elements with only weak fwd guarantee required, move to TSX!

```
#define __CMPXCHG_DBL(name, mb, rel, cl)                                \
static inline long                                                      \
__ll_sc__cmpxchg_double##name(unsigned long old1,                       \
                              unsigned long old2,                       \
                              unsigned long new1,                       \
                              unsigned long new2,                       \
                              volatile void *ptr)                       \
{                                                                       \
        unsigned long tmp, ret;                                         \
                                                                        \
        asm volatile("// __cmpxchg_double" #name "\n"                   \
        __LL_SC_FALLBACK(                                               \
        "       prfm    pstl1strm, %2\n"                                \
        "1:     ldxp    %0, %1, %2\n"                                   \
        "       eor     %0, %0, %3\n"                                   \
        "       eor     %1, %1, %4\n"                                   \
        "       orr     %1, %0, %1\n"                                   \
        "       cbnz    %1, 2f\n"                                       \
        "       st" #rel "xp    %w0, %5, %6, %2\n"                      \
        "       cbnz    %w0, 1b\n"                                      \
        "       " #mb "\n"                                              \
        "2:")                                                           \
        : "=&r" (tmp), "=&r" (ret), "+Q" (*(unsigned long *)ptr)        \
        : "r" (old1), "r" (old2), "r" (new1), "r" (new2)                \
        : cl);                                                          \
                                                                        \
        return ret;                                                     \
}
```

Conclusion
(Suggestions for LR/SC architectures)

- Give strict forward guarantee micro-arch implementation
- Use macro-op fusion to enhance atomic performance
- Support multi-word LR/SC primitive

Combo-lock solves existed problem of some over-flexible ISA definitions.



Thank you

Questions?



Linux
Plumbers Conference | Dublin, Ireland Sept. 12-14, 2022